Tarangini Workflow Suite

Complete Tool UML And Transaction Process Explanation

Billing, accounting, inventory, job order workflow, customer portal, security, backup, sync and deployment.

Purpose	Department-wise explanation PDF for owner review, operator training, customer portal flow, and end-to-end transaction verification.

How To Read This PDF

1. Sections 1-3 explain the complete application architecture, permissions and database entities.
2. Sections 4-11 explain customer intake, owner review, operator assignment, production, delivery and job billing.
3. Sections 12-19 explain billing, accounting, payments, inventory and bank transactions.
4. Sections 20-26 explain reports, customer portal, file retention, backup, security, sync and installer/update flow.
5. Section 27 is the transaction checklist: every major action, API, table, accounting effect, stock effect and audit effect.

Version: 1.0.0 Scope: Billing, accounting, inventory, production workflow, customer portal, backup, security, diagnostics and deployment.

This document is split into department-wise and transaction-wise UML diagrams. A single diagram for the whole tool would be too dense, so each process is shown separately and linked through common entities: `org_id`, `party_id`, `item_id`, `job_id`, `bill_id`, `payment_id`, `fy`, audit records and stock/accounting references.

1. Whole Tool Component UML

UML / mermaid definition

```
flowchart TB
  subgraph Clients["Client Interfaces"]
    Desktop["Desktop App / Electron Shell"]
    Browser["LAN Browser Clients"]
    CustomerIntake["Customer Intake QR Page"]
    CustomerPortal["Customer Job Portal"]
    FutureMobile["Future Android / PWA"]
  end

  subgraph Api["Express API Server"]
    Auth["Auth / PIN / Sessions"]
    Orgs["Company / Transaction Controls"]
    Masters["Parties / Items / Categories"]
    Billing["Billing / Invoices / POS"]
    Accounting["Accounting / Reports / Ledger"]
    Inventory["Stock / Materials"]
    Jobs["Job Order Workflow"]
    Intake["Customer Intake"]
    Portal["Customer Portal API"]
    Backup["Backup / Restore / Data Shift"]
    Advanced["Maintenance / Updates / Diagnostics"]
  end

  subgraph Services["Internal Services"]
    Db["SQLite WAL Database"]
    Postings["Accounting Posting Engine"]
    StockSvc["Stock Movement Engine"]
    AttachmentStore["Attachment Storage"]
    PdfAnalysis["Image/PDF Analyzer"]
    Retention["Attachment Retention Worker"]
    AutoBackup["Automatic Encrypted Backup"]
    Audit["Audit / Data Shift Logs"]
    Sync["Server Sent Events Sync"]
  end

  Desktop --> Api
  Browser --> Api
  CustomerIntake --> Intake
  CustomerPortal --> Portal
  FutureMobile -.future.-> Api

  Api --> Db
  Billing --> Postings
  Billing --> StockSvc
  Jobs --> Postings
  Jobs --> StockSvc
  Jobs --> AttachmentStore
  Intake --> AttachmentStore
```

```
  Portal --> AttachmentStore
  AttachmentStore --> PdfAnalysis
  Retention --> AttachmentStore
  Backup --> Db
  Backup --> Audit
  AutoBackup --> Db
  Api --> Sync
```

2. Main Actors And Permissions

UML / mermaid definition

```
flowchart LR
  Owner["Owner / owner1"] --> AllAccess["All companies, settings, backup, users, reports, billing, jobs"]
  Counter["Counter / Billing Staff"] --> CounterUse["Invoices, parties, customer intake review, job
    creation, delivery"]
  Finance["Finance User"] --> FinanceUse["Estimates, first bill, final bill, payments, accounting reports"]
  Operator["Operator"] --> OperatorUse["Assigned jobs, work reports, status updates, materials"]
  SeniorOperator["Senior Operator"] --> SeniorUse["Assign/forward jobs, operate production queues"]
  Engineer["Engineer"] --> EngineerUse["Technical work updates and reports"]
  Customer["Customer"] --> CustomerUse["QR order submission, upload files, status tracking, approvals,
    preview"]
  ClientPC["Client System"] --> LanUse["Uses main system database through API"]
```

3. High-Level Database Entity UML

UML / mermaid definition

```
erDiagram
  orgs ||--o{ users : access
  orgs ||--o{ parties : owns
  orgs ||--o{ items : owns
  orgs ||--o{ bills : owns
  orgs ||--o{ payments : owns
  orgs ||--o{ accounts : owns
  orgs ||--o{ journal_entries : owns
  orgs ||--o{ job_orders : owns

  parties ||--o{ bills : billed_to
  parties ||--o{ payments : pays
  parties ||--o{ job_orders : requests

  bills ||--o{ payment_allocations : settled_by
  payments ||--o{ payment_allocations : allocates
  bills ||--o{ stock_movements : sale_out
  bills ||--o{ journal_entries : posts

  accounts ||--o{ journal_lines : used_by
  journal_entries ||--o{ journal_lines : contains

  items ||--o{ stock_movements : moves
  items ||--o{ job_material_consumptions : consumed

  job_orders ||--o{ job_items : contains
  job_orders ||--o{ job_estimates : quoted
  job_orders ||--o{ job_assignments : assigned
  job_orders ||--o{ job_status_events : timeline
  job_orders ||--o{ job_work_reports : reports
```

```
  job_orders ||--o{ job_attachments : files
  job_orders ||--o{ job_material_consumptions : consumes
  job_orders ||--o{ job_additions : extra_work
  job_orders ||--o{ job_delivery_acknowledgements : delivered
  job_orders ||--o{ job_audit_events : audited
  job_orders }o--o| bills : pre_bill_id
  job_orders }o--o| bills : final_bill_id

  job_additions ||--o{ job_customer_approvals : approved_by
  job_customer_access_tokens }o--|| job_orders : opens_portal

  job_intake_requests ||--o{ job_intake_attachments : uploads
  job_intake_requests }o--o| parties : auto_party
  job_intake_requests }o--o| job_orders : converted_job
```

4. Complete Customer Intake Process

UML / mermaid definition

```
sequenceDiagram
  actor Customer
  participant QR as Customer Intake Page
  participant API as /api/job-intake
  participant DB as SQLite
  participant Analyzer as PDF/Image Analyzer
  participant Counter as Owner/Counter Review

  Customer->>QR: Scan QR / open company intake link
  QR->>API: GET /config?org_id
  API->>DB: Load org and service catalog
  DB-->>API: Services grouped by department
  API-->>QR: Service families and exact services

  Customer->>QR: Select Track Order or New Request
  Customer->>QR: Enter name, phone, email, address, service, instructions
  Customer->>QR: Upload PDF/images
  QR->>API: POST /analyze-file for preview analysis
  API->>Analyzer: Page count, color hints, dimensions, metadata
  Analyzer-->>API: Metadata
  API-->>QR: Analysis result

  Customer->>QR: Submit request with consent and files
  QR->>API: POST /submit with idempotency key
  API->>DB: Create/reuse party
  API->>DB: Insert job_intake_requests
  API->>DB: Insert job_intake_attachments
  API->>DB: Insert audit event
  API-->>QR: Request number / Order ID

  Counter->>API: GET /job-intake
  API->>DB: List submitted requests
  Counter->>API: GET /job-intake/:id
  Counter->>API: POST /job-intake/:id/convert
  API->>DB: Create job_orders, job_items, accepted estimate
  API->>DB: Copy intake files into job_attachments
  API->>DB: Link intake converted_job_id
  API-->>Counter: Job token
```

5. Job Order Lifecycle UML

UML / mermaid definition

```
stateDiagram-v2
  [*] --> WAITING: Job created
  WAITING --> ACCEPTED: Assignment accepted
  ACCEPTED --> IN_PROGRESS: Work starts
  ACCEPTED --> WAITING: Returned to waiting
  IN_PROGRESS --> WAITING_FOR_MATERIAL: Material unavailable
  WAITING_FOR_MATERIAL --> IN_PROGRESS: Material available
  IN_PROGRESS --> QUALITY_CHECK: Work completed by operator
  QUALITY_CHECK --> IN_PROGRESS: Rework needed
  QUALITY_CHECK --> COMPLETED: QC passed
  COMPLETED --> IN_PROGRESS: Reopen/rework
  COMPLETED --> READY_FOR_DELIVERY: Owner/counter ready
  READY_FOR_DELIVERY --> IN_PROGRESS: Return from delivery
  READY_FOR_DELIVERY --> DELIVERED: Invoice/delivery acknowledged
  DELIVERED --> [*]
```

6. Job Creation And Estimate Process

UML / mermaid definition

```
sequenceDiagram
  actor Counter
  participant JobUI as Job Dashboard
  participant Jobs as /api/jobs
  participant DB as SQLite
  participant Audit as Audit Log

  Counter->>JobUI: Create job for customer
  JobUI->>Jobs: GET /catalog
  Jobs->>DB: Ensure/load service catalog
  Jobs-->>JobUI: Categories, services

  Counter->>JobUI: Select party, service, commitment, estimate lines
  JobUI->>Jobs: POST /jobs
  Jobs->>DB: Generate job_token
  Jobs->>DB: Insert job_orders
  Jobs->>DB: Insert job_items
  Jobs->>DB: Insert job_estimates status ACCEPTED
  Jobs->>DB: Insert job_status_events WAITING
  Jobs->>Audit: CREATE job audit event
  Jobs-->>JobUI: Job detail with owner work order record
```

7. Assignment And Production Process

UML / mermaid definition

```
sequenceDiagram
  actor Owner
  actor Operator
  participant Jobs as /api/jobs
  participant DB as SQLite

  Owner->>Jobs: POST /jobs/:id/assign
```

```
  Jobs->>DB: Insert job_assignments PENDING_ACCEPTANCE
  Jobs->>DB: Audit ASSIGNMENT
  Jobs-->>Owner: Assignment created

  Operator->>Jobs: POST /jobs/assignments/:id/respond ACCEPT
  Jobs->>DB: Mark assignment ACCEPTED
  Jobs->>DB: If WAITING, status becomes ACCEPTED
  Jobs->>DB: Insert status event and audit

  Operator->>Jobs: POST /jobs/:id/status IN_PROGRESS
  Jobs->>DB: Validate transition and accepted assignment
  Jobs->>DB: Update job current_status
  Jobs->>DB: Insert status event

  Operator->>Jobs: POST /jobs/:id/reports
  Jobs->>DB: Insert job_work_reports
  Jobs->>DB: Audit work report
```

8. Materials Consumed And Stock Linkage

UML / mermaid definition

```
sequenceDiagram
  actor Operator
  participant Jobs as /api/jobs/:id/materials
  participant Stock as Stock Engine
  participant DB as SQLite

  Operator->>Jobs: Record item consumed, qty, notes
  Jobs->>DB: Validate active accepted assignment
  Jobs->>DB: Load inventory item
  Jobs->>Stock: Validate stock availability
  Jobs->>DB: Insert job_material_consumptions
  Jobs->>DB: Insert stock_movements source_type job_material qty_out
  Jobs->>DB: Audit MATERIAL CONSUME
  Jobs-->>Operator: Material record and warnings
```

9. Additional Work And Customer Approval

UML / mermaid definition

```
sequenceDiagram
  actor Operator
  actor Finance
  actor Customer
  participant Jobs as /api/jobs
  participant Portal as /api/job-portal
  participant DB as SQLite

  Operator->>Jobs: POST /jobs/:id/additions with reason/evidence
  Jobs->>DB: Insert job_additions PROPOSED
  Jobs->>DB: Store evidence attachments
  Jobs->>DB: Audit ADDITION PROPOSE

  Finance->>Jobs: PUT /jobs/additions/:id/price
  Jobs->>DB: Set customer_description, price, tax, total
  Jobs->>DB: State AWAITING_CUSTOMER
  Jobs->>DB: Store evidence hash
```

```
  Customer->>Portal: Open job portal link
  Portal->>DB: Show awaiting additions
  Customer->>Portal: Approve/reject with digital confirmation
  Portal->>DB: Insert job_customer_approvals
  Portal->>DB: Update addition APPROVED/REJECTED
  Portal->>DB: Audit actor_role CUSTOMER
```

10. First Bill / Pre-Bill Linkage

UML / mermaid definition

```
sequenceDiagram
  actor Owner
  participant JobUI as Job Dashboard
  participant Jobs as /api/jobs/:id/pre-bill
  participant DB as SQLite
  participant Accounting as Posting Engine
  participant CustomerPortal as Customer Portal

  Owner->>JobUI: Create First Bill from accepted estimate
  JobUI->>Jobs: POST /jobs/:id/pre-bill
  Jobs->>DB: Validate accepted estimate
  Jobs->>DB: Block if final_bill_id/pre_bill_id exists
  Jobs->>DB: Create SALE bill from estimate lines
  Jobs->>Accounting: postBill()
  Accounting->>DB: Create journal_entries and journal_lines
  Jobs->>DB: Update job_orders.pre_bill_id, pre_billed_at, financial_status BILLED
  Jobs->>DB: Audit PRE_BILL
  Jobs-->>JobUI: Bill number and linked job detail
  CustomerPortal->>DB: Shows linked invoice and payment status
```

11. Delivery And Final Invoice Process

UML / mermaid definition

```
sequenceDiagram
  actor Counter
  participant Jobs as /api/jobs/:id/deliver
  participant DB as SQLite
  participant Accounting as Posting Engine
  participant Retention as Attachment Retention

  Counter->>Jobs: Deliver job with receiver, remarks, payments
  Jobs->>DB: Require READY_FOR_DELIVERY and accepted estimate
  Jobs->>DB: Load approved additions and pre_bill_id

  alt No first bill exists
    Jobs->>DB: Create final SALE bill from estimate + approved additions
    Jobs->>Accounting: postBill()
  else First bill exists and total matches
    Jobs->>DB: Reuse pre_bill as final_bill_id
    Jobs->>DB: Audit PRE_BILL_REUSED
  else First bill exists but amount differs
    Jobs-->>Counter: Block duplicate invoice; request supplementary/correction handling
  end

  Jobs->>DB: Create delivery payment receipts if any
  Jobs->>Accounting: postPayment() for receipts
```

```
  Jobs->>DB: Insert job_delivery_acknowledgements
  Jobs->>DB: Update job status DELIVERED, financial status, final_bill_id
  Jobs->>DB: Insert status event and audit DELIVER
  Jobs->>Retention: Schedule 7-day attachment cleanup for non-archived files
```

12. Billing / Sales Invoice Transaction

UML / mermaid definition

```
sequenceDiagram
  actor Counter
  participant UI as Billing UI
  participant Bills as /api/bills
  participant DB as SQLite
  participant Stock as Stock Engine
  participant Accounting as Accounting Engine

  Counter->>UI: Enter sale / POS / project invoice
  UI->>Bills: GET /next-number
  Bills->>DB: Peek bill_sequences
  Bills-->>UI: Next invoice number

  Counter->>UI: Save invoice
  UI->>Bills: POST /bills
  Bills->>DB: Validate org, party, transaction controls, FY lock
  Bills->>DB: Calculate tax, round-off, split payments
  Bills->>DB: Insert bills
  Bills->>DB: Audit CREATE bill
  Bills->>Accounting: postBill()
  Accounting->>DB: Journal sales, taxes, receivable/cash/bank
  Bills->>Stock: replaceStockMovements for SALE
  Stock->>DB: stock_movements qty_out
  Bills-->>UI: Saved bill, payment status, stock warnings
```

13. Bill Edit, Delete, Convert And Correction

UML / mermaid definition

```
flowchart TB
  Start["Existing bill"] --> Edit["PUT /bills/:id"]
  Start --> Delete["DELETE /bills/:id owner only"]
  Start --> Convert["POST /bills/:id/convert"]
  Start --> Correction["POST /bills/:id/correction-request"]

  Edit --> Validate["Validate FY lock, controls, offline emergency rules"]
  Validate --> Repost["Update bill, repost accounting, replace stock movements"]

  Delete --> DeleteCheck["Owner + FY unlocked"]
  DeleteCheck --> DeletePost["Mark deleted, remove journal source, remove stock movements"]

  Convert --> ConvertCheck["Only QUOT / DC / PI"]
  ConvertCheck --> SaleBill["Create SALE bill, mark source converted"]
  SaleBill --> PostSale["Post accounting and stock out"]

  Correction --> Pending["Create pending correction request"]
  Pending --> Review["Owner reviews"]
  Review --> Reject["Reject"]
  Review --> Approve["Approve"]
```

```
  Approve --> CreditNote["Create credit note"]
  CreditNote --> NotePost["Post note and stock return"]
```

14. Payments And Settlement

UML / mermaid definition

```
sequenceDiagram
  actor Counter
  participant Reports as /api/payments
  participant DB as SQLite
  participant Accounting as Posting Engine

  Counter->>Reports: POST /payments received/paid
  Reports->>DB: Validate transaction controls and FY lock
  Reports->>DB: Generate payment_number
  Reports->>DB: Insert payments
  Reports->>DB: Insert payment_allocations if linked bills
  Reports->>Accounting: postPayment()
  Accounting->>DB: Journal cash/bank and receivable/payable
  Reports-->>Counter: Payment record

  Counter->>Reports: GET invoice/payment status
  Reports->>DB: billSettlement = immediate cash + allocations + credit notes
  Reports-->>Counter: paid / partly paid / unpaid / overdue
```

15. Purchase, Expense, Notes And Purchase Order

UML / mermaid definition

```
flowchart TB
  PO["Purchase Order"] --> POCreate["POST /advanced/purchase-orders"]
  POCreate --> POConvert["POST /advanced/purchase-orders/:id/convert"]
  POConvert --> Purchase["Purchase Voucher"]

  Purchase --> PurchasePost["POST /business/purchases"]
  PurchasePost --> PurchaseJournal["Post purchase / input GST / payable or cash-bank"]
  PurchasePost --> PurchaseStock["Stock movement qty_in"]

  Expense["Expense"] --> ExpensePost["POST /business/expenses"]
  ExpensePost --> ExpenseJournal["Post expense / GST / cash-bank"]

  Note["Credit/Debit Note"] --> NotePost["POST /business/notes"]
  NotePost --> NoteJournal["Post note accounting"]
  NotePost --> NoteStock["Stock movement if item rows"]
```

16. POS Shift And Held Bills

UML / mermaid definition

```
sequenceDiagram
  actor Cashier
  actor Owner
  participant Advanced as /api/advanced
  participant Bills as /api/bills
  participant DB as SQLite
```

```
  Cashier->>Advanced: POST /shifts/open
  Advanced->>DB: Insert pos_shifts OPEN
  Cashier->>Advanced: POST /held-bills
  Advanced->>DB: Save cart_json
  Cashier->>Advanced: DELETE /held-bills/:id
  Cashier->>Bills: POST /bills with shift_id
  Bills->>DB: Save sale linked to shift
  Cashier->>Advanced: PUT /shifts/:id/close
  Advanced->>DB: Store counted cash, variance, status closed
  Owner->>Advanced: PUT /shifts/:id/accept
  Advanced->>DB: Owner acceptance audit
```

17. Inventory And Stock UML

UML / mermaid definition

```
flowchart LR
  Opening["Item opening_stock"] --> StockReport["Stock report"]
  Sale["SALE bill"] --> Out["stock_movements qty_out source bill"]
  Return["Return / Credit note"] --> In1["stock_movements qty_in source note/return"]
  Purchase["Purchase"] --> In2["stock_movements qty_in source purchase"]
  JobMaterial["Job material consumption"] --> Out2["stock_movements qty_out source job_material"]
  Out --> StockReport
  In1 --> StockReport
  In2 --> StockReport
  Out2 --> StockReport
```

18. Accounting Posting UML

UML / mermaid definition

```
flowchart TB
  Bill["Bill / Sale"] --> PostBill["postBill()"]
  Payment["Payment / Receipt"] --> PostPayment["postPayment()"]
  Purchase["Purchase"] --> PostPurchase["postPurchase()"]
  Expense["Expense"] --> PostExpense["postExpense()"]
  Note["Credit/Debit Note"] --> PostNote["postNote()"]
  Manual["Manual Journal"] --> Journal["POST /accounting/journals"]

  PostBill --> JE["journal_entries"]
  PostPayment --> JE
  PostPurchase --> JE
  PostExpense --> JE
  PostNote --> JE
  Journal --> JE
  JE --> JL["journal_lines"]
  JL --> Trial["Trial Balance"]
  JL --> PL["Profit & Loss"]
  JL --> BS["Balance Sheet"]
  JL --> Ledger["Ledger Drilldown"]
  JL --> Receivables["Receivables / Payables"]
```

19. Bank Statement And Reconciliation

UML / mermaid definition

```
sequenceDiagram
```

```
  actor Accountant
  participant Business as /api/business
  participant DB as SQLite

  Accountant->>Business: POST /bank-statements/import XLSX/CSV rows
  Business->>DB: Insert bank_statement_imports with file_hash
  Business->>DB: Insert bank_statement_rows
  Business-->>Accountant: Imported count / duplicate protection

  Accountant->>Business: PUT /bank-statements/:rowId/match
  Business->>DB: Link bank row to journal_line_id
  Business->>DB: Insert/update bank_reconciliation
  Business-->>Accountant: Matched row
```

20. GST / Reports / Export Processes

UML / mermaid definition

```
flowchart TB
  Sales["Saved sales invoices"] --> GSTR1["GSTR-1 JSON / Portal data"]
  Purchases["Purchases and GSTR-2B import"] --> GSTR2B["GSTR-2B matching"]
  Parties["Party GST/state data"] --> GSTCalc["GST validation and state/POS logic"]
  Items["HSN and GST rates"] --> GSTCalc
  GSTCalc --> GSTR1
  GSTR1 --> Export["JSON / Report export"]
  Ledger["Journal lines"] --> Financial["Financial reports"]
  Bills["Bills/payments"] --> PartyStatement["Party statement"]
```

21. Customer Portal Job Status And Preview

UML / mermaid definition

```
sequenceDiagram
  actor Customer
  participant PortalPage as customer-job.html
  participant PortalAPI as /api/job-portal/:token
  participant DB as SQLite
  participant Files as Attachment Store

  Customer->>PortalPage: Open secure job token link
  PortalPage->>PortalAPI: GET /:token
  PortalAPI->>DB: Validate token hash, expiry, revoked_at
  PortalAPI->>DB: Load job, items, quotation, additions, visible attachments
  PortalAPI-->>PortalPage: Status, invoice link, payment status, attachments metadata

  Customer->>PortalPage: Preview visible file
  PortalPage->>PortalAPI: GET /attachments/:id/content
  PortalAPI->>DB: Check visible_to_customer
  alt File retained
    PortalAPI->>Files: Load bytes
    PortalAPI-->>PortalPage: Inline preview, no download button
  else File expired
    PortalAPI-->>PortalPage: 410 preview expired, metadata retained
  end

  Customer->>PortalPage: Upload extra file
  PortalPage->>PortalAPI: POST /uploads
  PortalAPI->>Files: Store file
```

```
  PortalAPI->>DB: Insert job_attachments and audit
```

22. Attachment Storage, Analysis, Retention And Archive

UML / mermaid definition

```
stateDiagram-v2
  [*] --> ACTIVE: Upload stored
  ACTIVE --> ACTIVE: Analysis pending/ready
  ACTIVE --> ARCHIVED: Owner moves to archive
  ARCHIVED --> ACTIVE: Owner removes archive
  ACTIVE --> DELETED: Retention worker after delivery + 7 days
  DELETED --> [*]: Metadata remains in database
```

UML / mermaid definition

```
sequenceDiagram
  participant Upload as Upload Endpoint
  participant Storage as Attachment Storage
  participant Analyzer as Analyzer Queue/Sync
  participant DB as SQLite
  participant Retention as Retention Worker
  actor Owner

  Upload->>Storage: persistAttachmentBytes()
  Storage-->>Upload: storage_path or inline base64
  Upload->>DB: Insert attachment row and metadata
  Upload->>Analyzer: Analyze image/PDF
  Analyzer->>DB: Update pixel/page/color metadata

  Owner->>DB: Mark archive_attachment=true
  DB-->>Owner: retention_state ARCHIVED

  Retention->>DB: Find ACTIVE expired non-archive files
  Retention->>Storage: Delete physical bytes
  Retention->>DB: retention_state DELETED, keep name/hash/metadata
```

23. Backup, Restore And Data Shift

UML / mermaid definition

```
sequenceDiagram
  actor Owner
  participant Backup as /api/backup
  participant DB as SQLite
  participant Shift as Data Shift Maps
  participant Crypto as AES-256-GCM Backup

  Owner->>Backup: GET /export org/fy
  Backup->>DB: Read orgs, parties, items, bills, payments, journals, jobs
  Backup->>DB: Read attachment metadata only
  Backup->>DB: Insert backup_log
  Backup-->>Owner: JSON backup

  Owner->>Backup: POST /import
  Backup->>Shift: Begin data_shift_batch
  Backup->>DB: Upsert orgs, parties, items, bills, payments
  Backup->>DB: Upsert jobs, estimates, assignments, attachments metadata
```

```
  Backup->>Shift: Record entity maps
  Backup->>Shift: Finish completed/failed

  Owner->>Backup: POST /run-automatic
  Backup->>DB: saveDB WAL checkpoint
  Backup->>Crypto: Encrypt SQLite DB
  Crypto-->>Backup: .tbe file
```

24. Security, Session And Role Flow

UML / mermaid definition

```
sequenceDiagram
  actor User
  participant UI as App UI
  participant Auth as /api/auth
  participant Middleware as Auth Middleware
  participant DB as SQLite

  User->>UI: Login username/password
  UI->>Auth: POST /login
  Auth->>DB: Verify bcrypt password and active user
  Auth->>DB: Create session/token
  Auth-->>UI: JWT/session user permissions

  UI->>Middleware: API request with bearer token
  Middleware->>DB: Validate session, org access, role permission
  alt Allowed
    Middleware-->>UI: Route response
  else Blocked
    Middleware-->>UI: 401/403/421/426
  end

  User->>UI: PIN unlock / change password / change PIN
  UI->>Auth: Secure auth endpoint
  Auth->>DB: Update hashes and audit
```

25. Sync Between Main And Client Systems

UML / mermaid definition

```
sequenceDiagram
  participant Main as Main System API
  participant ClientA as Client Browser A
  participant ClientB as Client Browser B
  participant DB as SQLite

  ClientA->>Main: POST/PUT/PATCH/DELETE transaction
  Main->>DB: Commit data
  Main->>Main: Increment revision
  Main-->>ClientA: Success
  Main-->>ClientB: SSE /api/sync/events revision changed
  ClientB->>Main: Refresh affected data
```

26. Update Package And Installer Process

UML / mermaid definition

```
flowchart TB
  Build["npm run dist"] --> Installer["NSIS Installer EXE"]
  Build --> Zip["Portable ZIP"]
  Installer --> Hash["SHA-256 recorded"]
  Zip --> Hash
  OwnerUpload["Owner uploads update package"] --> UpdatePackages["update_packages table"]
  UpdatePackages --> Verify["SHA-256 verification"]
  Verify --> Download["LAN clients download verified installer"]
  Download --> Install["Controlled installation"]
```

27. Full Transaction Matrix

Area	Transaction / Process	Main API	Main Tables	Accounting Impact	Stock Impact	Audit
Auth	Login	`POST /api/auth/login`	`users`, `sessions`	No	No	Session/login metadata
Auth	Change password/PIN	`/api/auth/change-*`	`users`, `audit_log`	No	No	Yes
Company	Create/update org	`/api/orgs`	`orgs`, `transaction_control_settings`	No	No	Yes
Masters	Party create/update	`/api/parties`	`parties`, `party_org_links`	Opening balance can post	No	Yes
Masters	Item create/update	`/api/items`	`items`, `item_categories`	No	Opening stock affects stock report	Yes
Billing	Sale/POS invoice	`POST /api/bills`	`bills`, `bill_sequences`	Sales/tax/cash/bank/receivable journal	SALE qty out	Yes
Billing	Edit invoice	`PUT /api/bills/:id`	`bills`	Repost journal	Replace stock movement	Yes
Billing	Delete invoice	`DELETE /api/bills/:id`	`bills`	Remove journal source	Remove stock movement	Yes
Billing	Convert quotation/DC/PI	`POST /api/bills/:id/convert`	`bills`	New SALE posting	SALE qty out	Yes
Billing	Correction request	`POST /api/bills/:id/correction-request`	`invoice_correction_requests`	No until approved	No until approved	Yes
Billing	Correction approval	`PUT /api/bills/corrections/:id/review`	`credit_debit_notes`	Credit note posting	Stock return where applicable	Yes
Payments	Receipt/payment	`POST /api/payments`	`payments`, `payment_allocations`	Cash/bank and receivable/payable	No	Yes
Purchase	Purchase voucher	`POST /api/business/purchases`	`purchases`	Purchase/input tax/payable journal	Qty in	Yes
Expense	Expense voucher	`POST /api/business/expenses`	`expenses`	Expense/GST/cash-bank journal	No	Yes
Notes	Credit/debit note	`POST /api/business/notes`	`credit_debit_notes`	Note journal	Optional stock movement	Yes
POS	Open/close shift	`/api/advanced/shifts`	`pos_shifts`	No direct	No	Yes
POS	Held bill	`/api/advanced/held-bills`	`held_bills`	No until invoice	No until invoice	Minimal
Job	Create job	`POST /api/jobs`	`job_orders`, `job_items`, `job_estimates`	No	No	Job audit
Job	First bill	`POST /api/jobs/:id/pre-bill`	`job_orders`, `bills`	SALE posting	No stock movement from job service lines	Job + bill audit
Job	Assign job	`POST /api/jobs/:id/assign`	`job_assignments`	No	No	Job audit
Job	Assignment response	`POST /api/jobs/assignments/:id/respond`	`job_assignments`, `job_status_events`	No	No	Job audit
Job	Status update	`POST /api/jobs/:id/status`	`job_orders`, `job_status_events`	No	No	Job audit
Job	Work report	`POST /api/jobs/:id/reports`	`job_work_reports`	No	No	Job audit
Job	Material consumed	`POST /api/jobs/:id/materials`	`job_material_consumptions`, `stock_movements`	No direct	Qty out	Job audit
Job	Additional work	`POST /api/jobs/:id/additions`	`job_additions`, `job_attachments`	No until billed	No	Job audit
Job	Price addition	`PUT /api/jobs/additions/:id/price`	`job_additions`	No until billed	No	Job audit
Job	Customer approval	`/api/job-portal/:token/additions/:id/decision`	`job_customer_approvals`, `job_additions`	No until billed	No	Customer audit
Job	Delivery	`POST /api/jobs/:id/deliver`	`job_delivery_acknowledgements`, `job_orders`, `bills`, `payments`	Final/reused bill and receipts	No extra unless invoice item stock exists	Job audit

Area	Transaction / Process	Main API	Main Tables	Accounting Impact	Stock Impact	Audit
Intake	Customer submit	`POST /api/job-intake/submit`	`job_intake_requests`, `job_intake_attachments`, `parties`	No	No	Intake/job audit
Intake	Convert intake	`POST /api/job-intake/:id/convert`	`job_orders`, `job_items`, `job_estimates`	No	No	Job audit
Portal	Customer upload	`POST /api/job-portal/:token/uploads`	`job_attachments`	No	No	Customer audit
Attachment	Archive/delete lifecycle	`PATCH /api/jobs/attachments/:id`, retention worker	`job_attachments`	No	No	Job audit
Accounting	Manual journal	`POST /api/accounting/journals`	`journal_entries`, `journal_lines`	Direct journal	No	Yes
Bank	Import statement	`POST /api/business/bank-statements/import`	`bank_statement_imports`, `bank_statement_rows`	No	No	Yes
Bank	Match statement	`PUT /api/business/bank-statements/:id/match`	`bank_reconciliation`	No	No	Yes
Backup	Manual export/import	`/api/backup/export`, `/api/backup/import`	All business/job tables, `data_shift_*`	Restored	Restored	Data shift audit
Backup	Automatic encrypted backup	`/api/backup/run-automatic`	SQLite DB file, `backup_log`	Database copy	Database copy	Yes
Maintenance	Integrity/repair	`/api/advanced/integrity`, `/api/advanced/integrity/repair`	Diagnostic tables	Rebuild possible	Rebuild possible	Yes
Updates	Update package	`/api/advanced/update-packages`	`update_packages`	No	No	Yes

28. Recommended Reading Order

1. Read diagrams 1-3 for the complete architecture.
2. Read diagrams 4-11 for customer/job production workflow.
3. Read diagrams 12-19 for billing/accounting/inventory transactions.
4. Read diagrams 20-26 for reporting, backup, security, sync and deployment.
5. Use the transaction matrix as the checklist for testing and department training.